

I'm working on a Pygame tug of war game. The player pulls by alternating the A and D keys.

Bug: when I mash A and D very fast, many of my presses are ignored, the rope barely moves, and the computer wins easily.

Please look at `handle_event()` in `game/game_engine.py`. First explain in simple words why fast or overlapping key presses get ignored. Then fix it so that every alternating press (A then D then A...) always registers as a pull, even when keys overlap. Pressing the same key twice in a row should still not count. Don't change anything else in the game.

Viewed `game_engine.py`:1-101

Viewed `main.py`:1-33

Searched for `"is_pull_locked"`

Listed directory `56_tug_of_war`

Viewed `README.md`:1-92

Edited `game_engine.py`

Ran command: ``python -m py_compile game/game_engine.py``

Viewed `game_engine.py`:1-96

Why Fast or Overlapping Presses Were Ignored

In the original

`[`handle_event()`](file:///c:/Users/swara/OneDrive/Desktop/tug_of_war/56_tug_of_war/game/game_engine.py#L26-L37)` implementation:

1. When you pressed `**A**`, ``is_pull_locked`` was set to ``True``.
2. The game would only unlock pulling (``is_pull_locked = False``) once it received a ``KEYUP`` event for the key you just pressed.
3. When rapidly alternating keys (mashing), fingers naturally overlap: you press `**D**` *before* completely lifting your finger off `**A**`.
4. Because ``is_pull_locked`` was still ``True`` during that press, the new key press was completely ignored. If key release order got out of sync, the lock would even deadlock input until the last key was released.

The Fix

To allow fluid mashing with key rollover while still requiring alternating presses:

- We track which key was pressed last (`self.last\_key`).
- Every `KEYDOWN` for **A** or **D** checks if `event.key != self.last\_key`. If it is different, it immediately counts as a pull and updates `self.last\_key`.
- If the player presses the same key twice in a row (e.g., A then A), it is ignored.
- We do not wait for a `KEYUP` event or lock pulling, allowing presses to register reliably even when keys overlap.

Here is the updated

[`handle\_event()`](file:///c:/Users/swara/OneDrive/Desktop/tug_of_war/56_tug_of_war/game/game_engine.py#L26-L37) method in [game/game_engine.py](file:///c:/Users/swara/OneDrive/Desktop/tug_of_war/56_tug_of_war/game/game_engine.py):

```
```python
def handle_event(self, event):
 if self.game_state != "PLAYING":
 if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
 self.reset()
 return

 if event.type == pygame.KEYDOWN:
 if event.key in (pygame.K_a, pygame.K_d):
 if event.key != self.last_key:
 self.rope.pull_left(1.0)
 self.last_key = event.key
```

...

Ran command: `python main.py`

Viewed game\_engine.py:59-77

Ran command: `python main.py`

The bug fix works. Now Task 2 from the README: dynamic AI panic surges.

Right now the computer pulls every 180 ms with a random strength between 0.7 and 1.2. Change it so that the closer the red marker gets to the player's winning line (rope.left\_win\_x), the more the computer panics:

- its pull interval gets shorter (faster pulls)
- its pull strength gets higher

Make it increase smoothly as the marker gets closer, not just switch on suddenly. Near the center the computer should behave the same as now.

Also show a red "PANIC!" text on screen while the computer is in its panic state, so it's visible in my demo video. Keep the game winnable, and don't change anything else.

Viewed rope.py:1-66

Viewed player.py:1-25

Searched for "computer\_pull\_cooldown"

Searched for "test"

Edited game\_engine.py

Ran command: `python -m py\_compile game/game\_engine.py`

Viewed game\_engine.py:1-119

### Changes Implemented for Task 2 (Dynamic AI Panic Surges)

In

[game/game\_engine.py](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py):

### 1. **Panic Calculation & State Tracking**:

- In

[update()](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L40-L65), when the rope marker is pulled past the center toward the player's winning line (`rope.left_win_x`), we compute a smooth `panic_factor` normalized from `0.0` (at center) to `1.0` (at `rope.left_win_x`).

- Near or to the right of the center line, `panic_factor = 0.0` and `is_panicking = False`.

### 2. **Smooth Scaling (Cooldown & Strength)**:

- **Pull Interval**: Smoothly scales down from `180 ms` (normal) to `120 ms` (maximum panic) via `180 - int(60 * panic_factor)`.

- **Pull Strength**: Minimum and maximum strengths scale smoothly from `[0.7, 1.2]` up to `[1.0, 1.6]` via `random.uniform(0.7 + 0.3 * panic_factor, 1.2 + 0.4 * panic_factor)`.

- Near the center, behavior remains identical to before (`cooldown = 180`, strength `[0.7, 1.2]`).

- The game remains winnable: steady, rapid alternating mashing from the player overcomes the surge to cross the line.

### 3. **Visual Feedback ('PANIC!' Text)**:

- In

[render()](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L77-L98), whenever `is_panicking` is `True` during active gameplay, a bright red `PANIC!` text is rendered directly above the computer puller anchor so it is clearly visible in your demo video.

- States are cleanly reset when restarting the game via

[reset()]([file:///c:/Users/swara/OneDrive/Desktop/tug\\_of\\_war/56\\_tug\\_of\\_war/game/game\\_engine.py#L67-L76](file:///c:/Users/swara/OneDrive/Desktop/tug_of_war/56_tug_of_war/game/game_engine.py#L67-L76)).

Task 2 works. Now Task 3 from the README: rope tension and puller leaning animations.

1. Rope: make the rope vibrate slightly (a small wobble) when the struggle is intense — for example when both sides have pulled recently or the computer is panicking. When nothing much is happening, let the rope sag a little in the middle instead of being a straight line.

2. Pullers: make both characters lean backwards based on who has the momentum. Whoever pulled more recently or is winning should lean back further, like they're pulling hard. The losing side should lean forward slightly.

Track momentum from the last few pulls, not just the current position. Keep the animations smooth and subtle, and keep all the existing gameplay exactly the same.

Viewed player.py:1-25

Edited player.py

Viewed rope.py:1-66

Edited rope.py

Viewed game\_engine.py:1-85

Edited game\_engine.py

Ran command: `python -m py\_compile game/player.py game/rope.py game/game\_engine.py`

Viewed player.py:1-34

Viewed rope.py:1-96

Viewed game\_engine.py:1-161

### Changes Implemented for Task 3 (Rope Tension & Puller Leaning)

#### 1. Dynamic Rope Sag & Vibration Wobble

In

[game/rope.py](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/rope.py):

- **\*\*Sag vs. Vibration Formula\*\***

([`Rope.get\_y\_at()`](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/rope.py#L20-L36)):

- We use a parabolic envelope  $E(u) = 4u(1-u)$  pinned at both anchor ends ( $x=60$  and  $x=\text{width}-60$ ).
- **Relaxed State** (`tension` near 0`): The rope curves smoothly downward with a natural sag in the center (`sag_amount = 14.0px``).
- **High Struggle State** (`tension` near 1`): The sag pulls taut and a high-frequency sine vibration wobble is applied (`wobble_amplitude = 2.5px``).
- **Dynamic Flag & Multi-Segment Rope**  
(`[Rope.render()]`(file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/rope.py#L56-L96)):
- Renders the rope as connected curve points using `pygame.draw.lines()`.
- The center flag's vertical position smoothly tracks the rope curve at `self.marker_x``.

---

## #### 2. Puller Leaning Postures

In

[game/player.py](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/player.py):

- **Leaning Geometry**  
(`[Puller.render()]`(file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/player.py#L16-L34)):
- Feet remain planted at base `(x, y + 35)``.
- The torso is drawn as a tilted polygon based on `facing_direction`` and continuous `lean`` factor.
- Positive lean tilts the upper body and head **backward** away from the rope (digging heels in), while negative lean tilts them **forward** toward the opponent.

---

## #### 3. Momentum & Struggle Intensity Tracking

In

[game/game\_engine.py](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py):

- **\*\*Struggle Intensity\*\***

([GameEngine.update()](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L68-L80)):

- Tracks decaying `player\_pull\_activity` and `computer\_pull\_activity` over time.
- Computes struggle tension combining mutual pull intensity (`min(player, computer)`), recent activity, and AI panic.
- Smoothly interpolates `rope.tension` between resting sag (0.0) and high-tension vibration (1.0).

- **\*\*Momentum & Leaning\*\***

([GameEngine.update()](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L82-L98)):

- Blends the recent pull rate differential with the board position advantage into a smooth `momentum` value.
- The side with momentum leans back hard to pull, while the losing side is pulled forward slightly.
- Interpolated per-frame (`0.1` lerp factor`) for natural, fluid motion.

- States reset cleanly upon pressing `[R]` in

[reset()](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L104-L118).

Ran command: `python main.py`

Ran command: `python main.py`

Ran command: `python main.py`

Viewed game\_engine.py:4-21

Task 3 works. Now Task 4 from the README: a match timer and sudden death mode.

1. Show a live match timer at the top of the screen (for example "0:12"). Place it so it doesn't overlap the "Alternate [A] and [D]" instruction text.

2. If nobody has won after 45 seconds, switch to "SUDDEN DEATH": every pull from both the player and the computer becomes twice as strong. Show "SUDDEN DEATH!" clearly on screen while it's active.

3. The timer should stop when the game ends, and pressing R should reset the timer and turn sudden death off.

Keep all the earlier fixes and features working exactly as they are.

Viewed game\_engine.py:15-45

Viewed game\_engine.py:100-161

Viewed game\_engine.py:15-161

Edited game\_engine.py

Ran command: `python -m py\_compile game/game\_engine.py`

Viewed game\_engine.py:1-189

### Changes Implemented for Task 4 (Match Timer & Sudden Death Mode)

In

[game/game\_engine.py](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py):

#### 1. Live Match Timer

- **Tracking & Formatting**

([`GameEngine.update()`](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L55-L58),  
[`GameEngine.render()`](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L144-L152)):

- Tracks `elapsed\_time` from `start\_time` while `game\_state == "PLAYING"`.
- Formats live time as `"M:SS"` (e.g., `"0:12"`).
- Positioned at `y = 12` (centered horizontally), sitting above the instructions text at `y = 40` with zero overlap.
- When the game ends, `elapsed\_time` automatically freezes at the final winning time.

---

## #### 2. Sudden Death Mode (45-Second Threshold)

### - **Activation & Visual Indicator**

([`GameEngine.update()`](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_Fast/game/game\_engine.py#L57),  
[`GameEngine.render()`](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L158-L161)):

- Triggers when `elapsed\_time >= 45000` (45 seconds).

- Renders a prominent red **``SUDDEN DEATH!``** banner at `y = 70` and highlights the timer in red.

### - **2x Pull Strength Multiplier**:

#### - **Player**

([`GameEngine.handle\_event()`](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L46-L50)): Every alternating keystroke pulls with `2.0` strength (`24px` instead of `12px`).

#### - **Computer**

([`GameEngine.update()`](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L73-L78)): Computer pulls with `2.0x` strength on both baseline and panic surge pulls.

---

## #### 3. Match Reset

### - In

([`GameEngine.reset()`](file:///c:/Users/swara/OneDrive/Desktop/tug\_of\_war/56\_tug\_of\_war/game/game\_engine.py#L116-L133), pressing `[R]` resets `start\_time`, clears `elapsed\_time` to 0, and turns `is\_sudden\_death` off. All earlier features (input synchronization, AI panic, leaning animations, and rope sag/vibration) remain fully intact.